2. 8-Queen Problem (Backtracking Approach)

```
def is_safe(board, row, col):
    for i in range(col):
        if board[row][i] == 1: return False
    for i, j in zip(range(row, -1, -1), range(col, -1, -1)):
        if board[i][j] == 1: return False
    for i, j in zip(range(row, 8, 1), range(col, -1, -1)):
        if board[i][j] == 1: return False
    return True

def solve_8_queens_util(board, col):
    if col >= 8: return True
    for i in range(8):
        if is_safe(board, i, col):
            board[i][col] = 1
            if solve_8_queens_util(board, col + 1): return True
            board[i][col] = 0
    return False

if __name__ == "__main__":
    chessboard = [[0]*8 for _ in range(8)]
    if solve_8_queens_util(chessboard, 0):
        print("Valid configuration for 8-Queens:")
        for r in chessboard:
            print(" ".join("Q" if x == 1 else "." for x in r))
```

main.py | Visualize | Run ▶

```python
def is_safe(board, row, col):
    # Check this row on left side
    for i in range(col):
        if board[row][i] == 1:
            return False

    # Check upper diagonal on left side
    for i, j in zip(range(row, -1, -1), range(col, -1, -1)):
        if board[i][j] == 1:
            return False

    # Check lower diagonal on left side
    for i, j in zip(range(row, 8), range(col, -1, -1)):
        if board[i][j] == 1:
            return False

    return True

def solve_8_queens_util(board, col):
    if col >= 8:
        return True
```

Output

```
One Valid 8-Queens Configuration:
Q . . . . . . .
. . . . . . Q .
. . . . Q . . .
. . . . . . . Q
. Q . . . . . .
. . . Q . . . .
. . . . . Q . .
. . Q . . . . .
```